

一文搞懂 Harness Engineering：六层架构、上下文管理与一线团队实战

👤 Guide 📄 AI 应用开发 📖 约 8775 字 ⌚ 大约 29 分钟

最近大半年，很多开发者都有同感：明明用的是最贵的模型，Agent 跑起来还是各种拉胯——重复犯错、做到一半放弃、越跑越蠢。换了更强的模型，效果也没好到哪去。

原因不在模型。[Can.ac](#) 做了个实验直接证明了这一点：同一个模型，只换了文件编辑接口的调用方式，编码基准分数从 6.7% 直接跳到 68.3%。模型没变，变的是外围的那套系统。

Harness Engineering 正在成为 AI Agent 开发圈的高频词。Mitchell Hashimoto 在博客里用了这个说法（他原话是“我不知道业界有没有公认的术语，我自己管这叫 harness engineering”），OpenAI 几天后发了一篇百万行代码的实验报告，Birgitta Böckeler 在 Martin Fowler 网站上写了深度分析，Anthropic 在三月份又放出了全新的多智能体架构设计。几周之内，Harness 成了讨论 AI Agent 开发绕不开的概念。

今天这篇文章就来系统梳理 Harness Engineering 的核心概念和工程方法，帮你搞清楚：**决定 Agent 表现的天花板，到底在哪里**。本文接近 1.3w 字，建议收藏，你将搞懂：

1. **Harness 到底是什么**：为什么说“你不是模型，那你就是 Harness”？Agent = Model + Harness 这个公式怎么理解？和 Prompt Engineering、Context Engineering 是什么关系？六层架构长什么样？
2. 🌟 **为什么瓶颈不在模型而在 Harness**：同一个模型只换了接口格式，分数从 6.7% 跳到 68.3%？上下文用到 40% Agent 就开始变蠢？
3. 🌟 **从零搭建 Harness 的行动清单**：Po/P1/P2 三个优先级，按需取用。
4. 🌟 **一线团队实战案例（附录）**：OpenAI 三人五月百万行零手写、Anthropic 的 GAN 式三智能体架构和 context resets 交接棒策略、Stripe 每周 1300+ 无人值守 PR、Mitchell Hashimoto 的六步进阶。

📌 **系列阅读**：本文是 AI Agent 系列的一部分，相关文章：

- [AI Agent 核心概念：Agent Loop、Context Engineering、Tools 注册](#)
- [Agent Skills 详解：是什么？怎么用？和 Prompt、MCP 有什么区别？](#)
- [万字拆解 MCP，附带工程实践](#)

★ Harness 核心概念

Harness 到底是什么？

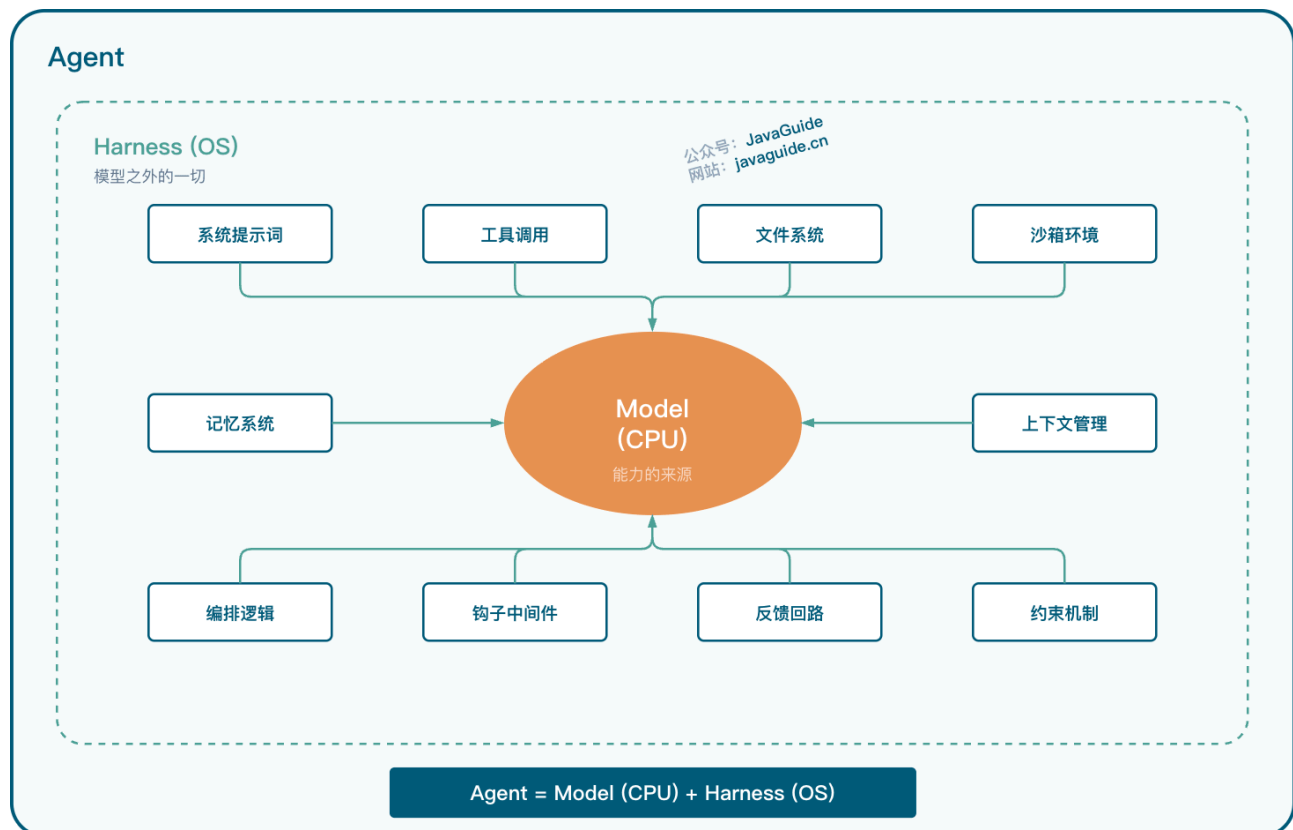
一句话：**Agent = Model + Harness**。你不是模型，那你就是 **Harness**。

听起来有点绝对？但仔细想想，它确实抓住了关键。

Harness 就是模型之外的一切——系统提示词、工具调用、文件系统、沙箱环境、编排逻辑、钩子中间件、反馈回路、约束机制。模型本身只是能力的来源，只有通过 Harness 把状态、工具、反馈和约束串起来，它才真正变成一个 Agent。

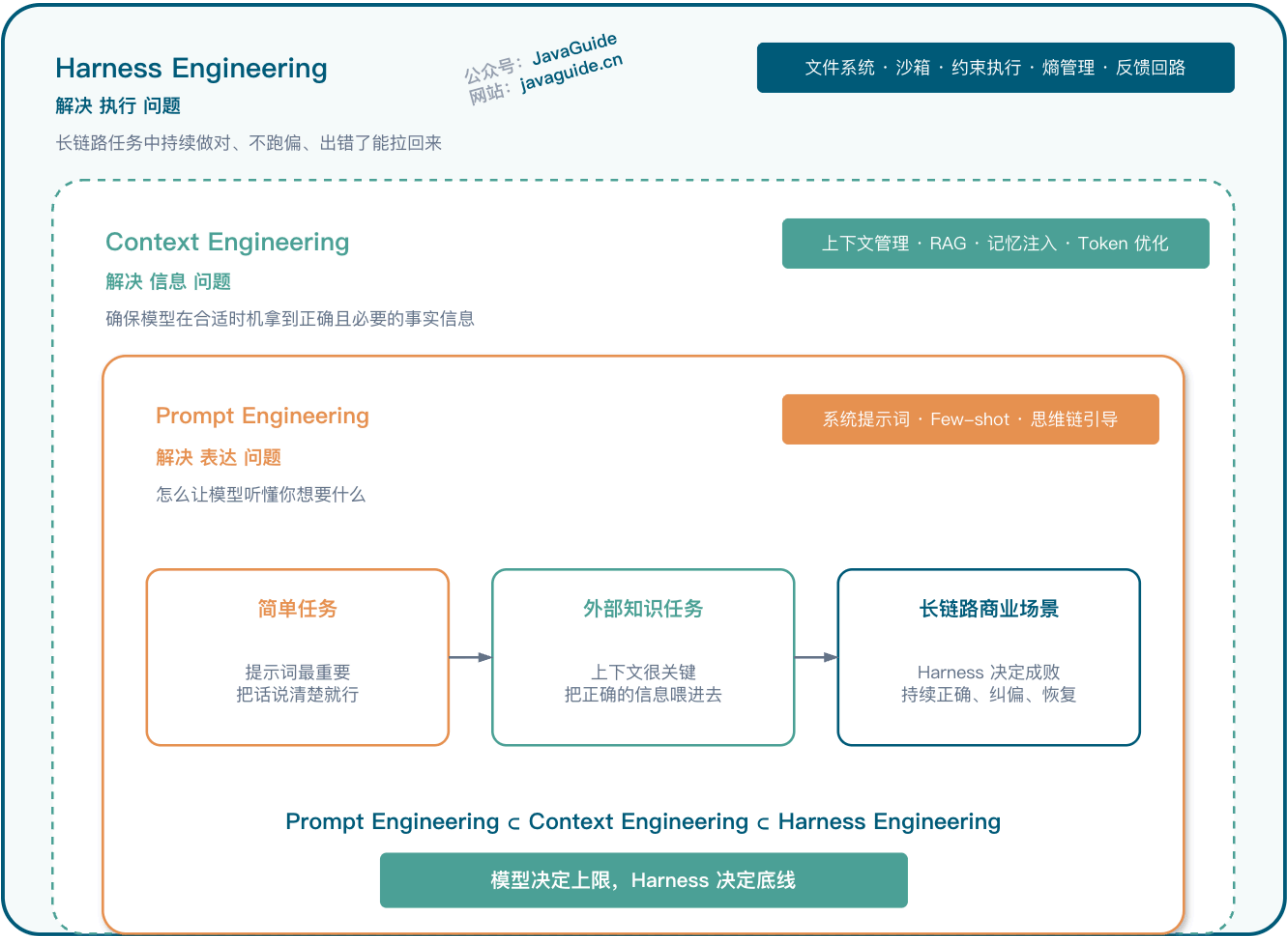
LangChain 的 Vivek Trivedi 在《The Anatomy of an Agent Harness》里把这个定义讲得很清楚：**先搞清楚模型负责什么，剩下的系统要补什么，用这条线把整个系统切开。**

打个比方：模型是 CPU，Harness 是操作系统。CPU 再强，OS 拉胯也白搭。你买了最新款 M5 芯片，装了个崩溃不断的系统，体验还不如老芯片配稳定的 OS。



Harness 和 Prompt/Context Engineering 是什么关系？

三者不是并列关系，而是嵌套关系。更重要的是，**每一层解决的是完全不同的问题**：



层级	解决的核心问题	关注点	典型工作
Prompt Engineering	表达——怎么写好指令	塑造局部概率空间，让模型听懂意图	系统提示词设计、Few-shot 示例、思维链引导
Context Engineering	信息——给 Agent 看什么	确保模型在合适的时机拿到正确且必要的事实信息	上下文管理、RAG、记忆注入、Token 优化
Harness Engineering	执行——整个系统怎么防崩、怎么量化、怎么持续运转	长链路任务中的持续正确、偏差纠正、故障恢复	文件系统、沙箱、约束执行、熵管理、反馈回路

简单任务里，提示词最重要——你把话说清楚就行；依赖外部知识的任务里，上下文很关键——你得把正确的信息喂进去；但在长链路、可执行、低容错的真实商业场景里，Harness 才是决定成败的东西。一线团队的重心都放在了 Harness 上，原因就在这。

Harness 包含哪些组件？

理解 Harness 的最好方式，不是直接看它包含什么，而是看模型做不到什么。不管大模型看起来多能干，本质就是一个文本（或图像、音频）进、文本出的函数。

模型做不到的，就是 **Harness** 要补的：

模型做不到	Harness 怎么补	核心组件
记住多轮对话历史	维护对话历史，每次请求时拼进上下文	记忆系统
执行代码、跑命令	提供 Bash + 代码执行环境	通用执行环境
获取实时信息（新库版本、API 变化）	Web Search、MCP 工具	外部知识获取
操作文件和环境	文件系统抽象 + Git 版本控制	文件系统
知道自己做对了没有	沙箱环境 + 测试工具 + 浏览器自动化	验证闭环
在长任务中保持连贯	上下文压缩、记忆文件、进度追踪	上下文管理

把这些“模型做不了但你希望 Agent 能做到”的事情一个个补上，就得到了 Harness 的核心组件。LangChain 把这件事拆解为五个子系统：文件系统（持久化）、Bash 执行（通用工具）、沙箱环境（安全隔离）、记忆机制（跨会话积累）、上下文压缩（对抗衰减）。

Harness 进阶

★ 一个成熟的 Harness 长什么样？

上面对组件的理解是“缺什么补什么”的思路。但如果从系统设计的角度看，一个成熟的 Harness 其实有清晰的层次结构。

我在 YouTube 上看到过一个六层体系的分享，觉得这个框架把 Harness 的全貌描绘得比较完整：



层级	名称	解决什么问题	关键设计
L1	信息边界层	Agent 该知道什么、不该知道什么	定义角色与目标，裁剪无关信息，结构化组织任务状态
L2	工具系统层	Agent 怎么跟外部世界交互	工具的选拔、调用时机、结果的提炼与反馈
L3	执行编排层	多步骤任务怎么串起来	让模型像人一样走完“理解目标→判断信息→分析→生成→检查”的完整轨道

层级	名称	解决什么问题	关键设计
L4	记忆与状态层	长任务中间结果怎么管	独立管理当前任务状态、中间产物和长期记忆，防止系统混乱
L5	评估与观测层	Agent 怎么知道自己做对了没有	建立独立于生成过程的验证机制，让 Agent 具备“自知之明”
L6	约束、校验与恢复层	出错了怎么办	预设规则拦截错误，失败时（API 超时、格式混乱）提供重试或回滚机制

可以类比成给一个新手员工搭建的完整工作环境。L1 是岗位说明书（告诉 ta 该关注什么），L2 是办公工具（给 ta 用什么干活），L3 是标准操作流程（按什么步骤做事），L4 是项目管理系统和笔记本（怎么记住做过的事），L5 是质检流程（怎么检验做对了没有），L6 是红线规则和应急预案（什么事绝对不能做、出了事怎么补救）。

这个六层架构最大的价值在于——它不是简单的功能堆叠，而是一个从“定义边界”到“兜底恢复”的完整闭环。附录中一线团队的实践也印证了这一点：他们的做法都可以映射到这六层里。

 **注意：**不要试图一开始就搭齐六层。从 L1（信息边界）和 L6（约束与恢复）入手，这两层投入产出比最高。L1 决定了 Agent 知道该干什么，L6 决定了它搞砸了能不能拉回来。中间的层次随着项目复杂度增长逐步补齐。

为什么瓶颈不在模型而在 Harness？

说实话，第一次看到这个结论的时候我也觉得反直觉——不是应该等更强的模型出来就好了吗？但数据确实不支持这个想法。OpenAI、Anthropic、Stripe、LangChain、[Can.ac](#) 的实验数据指向同一个结论：**基础设施才是瓶颈，而非智能水平。**

 **常见误区：**很多团队一遇到 Agent 表现不好，第一反应是“换更强的模型”或“调整提示词”。但 [Can.ac](#) 的实验证明，同一模型只换了工具调用格式，效果就能差十倍。**瓶颈大概率不在模型智能水平，而在 Harness 的基础设施质量。**

LangChain 那边也印证了这个结论：他们优化了 Agent 运行环境（文档组织方式、验证回路、追踪系统），在 Terminal Bench 2.0 上从全球第 30 名升到第 5 名，得分从 52.8% 提升到 66.5%。模型没换，Harness 换了。

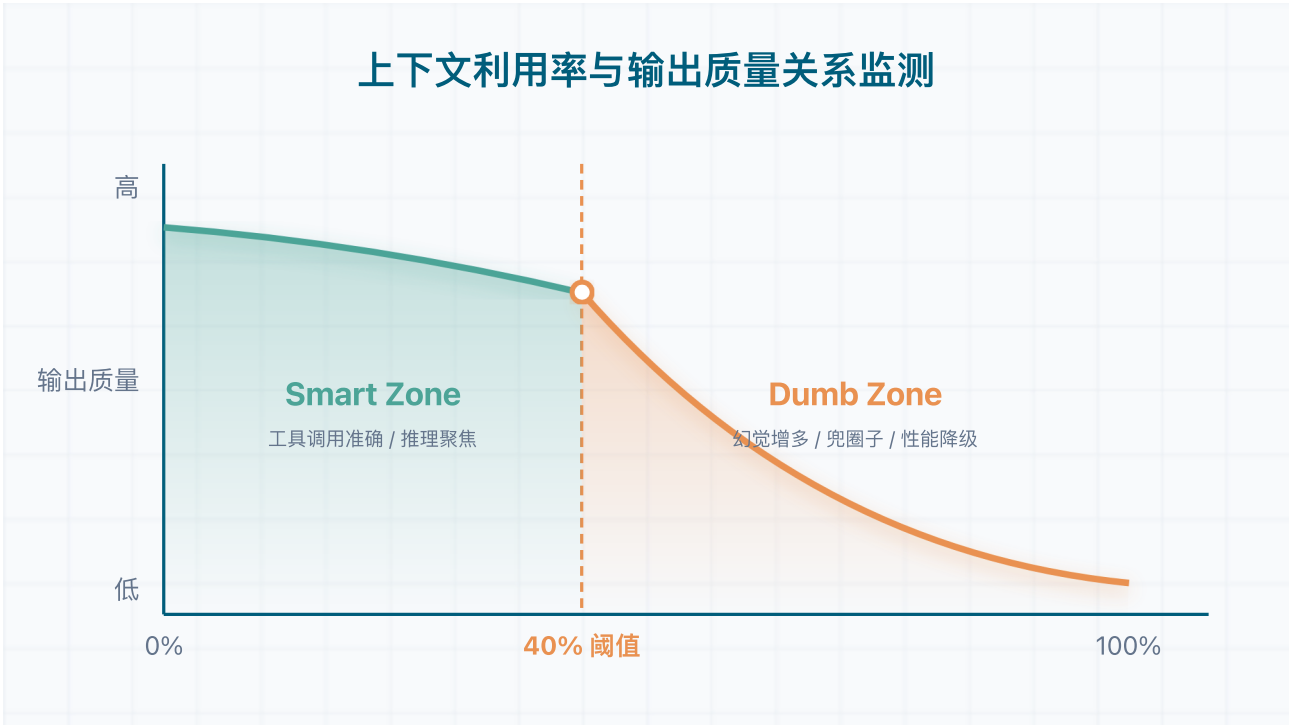
 **一个值得注意的发现：**

LangChain 还指出了一个 model-harness 耦合问题——当前的 Agent 产品（如 Claude Code、Codex）是模型和 Harness 一起训练的，这导致一种过拟合：**换了工具逻辑后模型表现会变差。**

他们在 Terminal Bench 2.0 排行榜上观察到，Opus 在 Claude Code 中的 Harness 下的得分，远低于它在其他 Harness 中的得分。结论是："the best harness for your task is not necessarily the one a model was post-trained with"——为你的任务选择 Harness 时，不要被模型的默认 Harness 束缚。

★ 为什么上下文喂越多，Agent 反而越蠢？

Dex Horthy 观察到一个现象：168K token 的上下文窗口，用到大约 40% 的时候，Agent 的输出质量就开始明显下降。



区间	占比	表现
Smart Zone	0 - ~40%	推理聚焦、工具调用准确、代码质量高
Dumb Zone	超过 ~40%	幻觉增多、兜圈子、格式混乱、低质量代码

Anthropic 在自己的实践中也碰到了类似的问题，他们叫“上下文焦虑”：Sonnet 4.5 在上下文快填满时会变得犹豫，倾向于提前收工——哪怕任务还没做完。光靠压缩不够，他们最终的做法是直接清空上下文窗口，但通过结构化的交接文档把关键状态留下来（详见附录中 Anthropic 的 context resets 策略）。

你的目标不是给 Agent 塞更多信息，而是让它在任何时候都运行在干净、相关的上下文里。一线团队的实践都围绕着“渐进式披露”和“分层管理”在做，背后的原因就是在这个 40% 阈值。

 **工程视角：**在生产环境中监控上下文利用率是第一优先级。建议设置 40% 阈值告警——当 Agent 的上下文占用超过这个比例时，就应该触发上下文压缩或任务交接。等到 Agent 已经变蠢了再处理就晚了。

★ 如果你要开始搭 Harness，应该从哪里入手？

综合一线团队的实践经验（详见附录），梳理了一个按优先级的行动路线。你不需要一开始就把所有东西都搞齐，先把 Po 做了效果就会很明显。

Po：不用犹豫，立即可以做

行动	为什么	参考实践
创建 AGENTS.md 并持续维护	Agent 每次启动自动加载，犯错就更新，形成反馈循环	Hashimoto 每一行对应一个历史失败案例
构建自定义 Linter + 修复指令	错误消息里直接告诉 Agent 怎么改，纠错的同时在“教”	OpenAI 的 Linter 报错自带修复方法
把团队知识放进仓库	写在 Slack/Wiki/Docs 里的知识对 Agent 等于不存在	OpenAI 以仓库为唯一事实源

 **常见误区：**很多团队把 AGENTS.md 当成“超级 System Prompt”来写，恨不得把所有规则塞进一个文件。结果上下文窗口被撑爆，Agent 反而更蠢了。正确做法是像 OpenAI 一样——AGENTS.md 只当目录用（约 100 行），详细规则放在子文档中按需加载。

P1：Po 做完之后，可以考虑这些

行动	为什么	参考实践
分层管理上下文	不要把所有东西塞进一个文件，渐进式披露	OpenAI <u>AGENTS.md</u> 当目录用（约 100 行）
建立进度文件和功能列表	JSON 格式追踪功能状态，Agent 不太会乱改结构化数据	Anthropic 初始化 Agent + 编码 Agent 两阶段

行动	为什么	参考实践
给 Agent 端到端验证能力	浏览器自动化让 Agent 能像用户一样验证功能	Anthropic 用 Playwright/Puppeteer MCP
控制上下文利用率	尽量不超过 40%，增量执行	Dex Horthy 的 Smart Zone / Dumb Zone

P2：有余力再考虑

行动	为什么	参考实践
Agent 专业化分工	每个 Agent 携带更少无关信息，留在 Smart Zone	Carlini 的去重/优化/文档 Agent
定期垃圾回收	确保清理速度跟得上生成速度	OpenAI 的后台清理 Agent
可观测性集成	把“性能优化”从玄学变成可度量的工作	OpenAI 接入 Chrome DevTools

你的 Harness 到哪个阶段了？

阶段	特征	工程师角色
Level 0：无 Harness	直接给 Agent prompt，无结构化约束	手动写代码 + 偶尔使用 AI
Level 1：基础约束	AGENTS.md + 基础 Linter + 手动测试	主要写代码，AI 辅助
Level 2：反馈回路	CI/CD 集成 + 自动化测试 + 进度追踪	规划 + 审查为主
Level 3：专业化 Agent	多 Agent 分工 + 分层上下文 + 持久化记忆	环境设计 + 管理为主
Level 4：自治循环	无人值守并行化 + 自动化熵管理 + 自修复	架构师 + 质量把关者

面试准备要点

Harness Engineering 相关的高频面试问题整理在下面，方便你快速回顾：

基础概念

问题	核心回答
Harness 是什么？	模型之外的一切——系统提示词、工具调用、文件系统、沙箱、编排逻辑、约束机制。 Agent = Model + Harness。
Harness 和 Prompt Engineering、Context Engineering 的关系？	嵌套关系：Prompt ⊂ Context ⊂ Harness。三者分别解决表达、信息、执行三个层面的问题。
为什么瓶颈不在模型而在 Harness？	Can.ac 实验证明同一模型只换工具调用格式，分数从 6.7% 跳到 68.3%。基础设施质量决定了模型能力的实际发挥。

架构设计

问题	核心回答
Harness 六层架构是什么？	L1 信息边界 → L2 工具系统 → L3 执行编排 → L4 记忆与状态 → L5 评估与观测 → L6 约束校验与恢复。从“定义边界”到“兜底恢复”的完整闭环。
上下文管理有什么经验法则？	利用率控制在 40% 以内。超过后 Agent 质量明显下降（幻觉增多、兜圈子）。策略是压缩或交接，不是继续塞信息。
单 Agent 还是多 Agent？	规模决定。小项目单 Agent 够用（Hashimoto 模式），大项目几乎必然需要专业化分工（Carlini 用 16 个并行 Agent）。

实战方案

问题	核心回答
OpenAI 的 Harness 实践核心是什么？	五大方法论：地图式文档（渐进式披露）、机械化约束（自定义 Linter）、可观测性接入、熵管理（定期垃圾回收）、仓库即事实源。
Anthropic 如何解决上下文焦虑？	Context resets 策略：不压缩，而是启动一个全新“干净”的 Agent，通过结构化交接文档恢复状态。类似重启进程解决内存泄漏。
从零搭 Harness 先做什么？	Po：创建 <u>AGENTS.md</u> + 自定义 Linter + 团队知识仓库化。投入产出比最高。

还没有答案的问题

Harness Engineering 是一个快速发展的领域，仍有许多未解的问题。了解这些“不知道”同样重要——面试时能展现你的思考深度。

问题	现状	谁在关注
棕地项目怎么改造？	所有公开案例全是绿地项目，零方法论	Böckeler：比作“在从没用过静态分析的代码库上跑静态分析”。她还提出“Ambient Affordances”概念：环境本身的结构特性（类型系统、模块边界、框架抽象）决定了 Harness 能做多好
怎么验证 Agent 做对了事？	大家擅长“约束不做错事”，但“验证做对了事”远未解决	Böckeler 批评：用 AI 生成的测试来验证 AI 生成的代码，本质上是“用同一双眼睛检查自己的作业”——"that's not good enough yet"
AI 生成代码的长期可维护性？	LLM 代码经常重新实现已有功能，长期效果未知	Greg Brockman 提出至今无人回答
Harness 该做厚还是做薄？	Manus 五次重写越做越简单 vs OpenAI 五个月越做越复杂	场景决定：通用产品追求最小化，特定产品可以高度定制。而且随着模型变强，已有 Harness 应该定期简化（Anthropic 实测验证）

问题	现状	谁在关注
单 Agent 还是多 Agent?	Hashimoto 坚持单 Agent vs Carlini 用 16 个并行 Agent	规模决定：小项目单 Agent 够用，大项目几乎必然需要专业化

绿地项目和棕地项目是软件工程里的经典比喻：

- 绿地项目（Greenfield）：从零开始的新项目，没有历史包袱。就像在一片空地上盖房子，想怎么设计都行。
- 棕地项目（Brownfield）：在已有代码库上改造，有历史架构、技术债、遗留逻辑的约束。就像在老旧城区搞翻新，到处是管线不能随便动。

OpenAI、Anthropic、Stripe、Hashimoto 这些成功案例，全部是在全新项目上从零搭 Harness。但现实中绝大多数团队面对的是已经跑了多年的代码库——怎么把 Harness 引入一个十年历史、没有架构约束、到处是技术债的项目？目前没有任何公开方法论。

总结

一句话概括 Harness Engineering 做的事情：**承认模型有边界，然后把边界之外的需求一个个工程化地补上。**

有一句话我特别认同：**模型决定了系统的上限，Harness 决定了系统的底线。**

在简单任务中提示词最重要，在依赖外部知识的任务中上下文很关键，但在长链路、可执行、低容错的真实商业场景中，Harness 才是 AI 稳定落地的前提条件。

如果只记一句话：**模型决定上限，Harness 决定底线。与其纠结选哪个模型，不如先把 Harness 搭好。**

附录：一线团队实战案例

OpenAI、Anthropic、Stripe、Mitchell Hashimoto、Martin Fowler，这五个团队/个人的实践从不同角度揭示了 Harness 设计中容易被忽略的问题。放在一起看会更有感觉——你会发现大家遇到的坑和总结出的经验，惊人地一致。

OpenAI：三个人、五个月、一百万行、零手写代码

先看数据：

指标	数值
团队规模	3 名工程师（后扩至 7 人）
持续时间	5 个月（2025 年 8 月起）
代码规模	约 100 万行
手写代码	0 行（设计约束）
合并 PR 数	约 1,500 个
日均 PR/人	3.5 个
效率提升	约 10 倍

比数字更有意思的是他们总结出来的五大方法论。

给 Agent 一张地图，而不是一本千页手册

OpenAI 的 AGENTS.md 只有大约 100 行，作用类似于目录，指向 docs/ 目录下更深层的设计文档、架构图、执行计划和质量评级。这是**渐进式披露**的实际运用——先把最关键的信息放进来，需要什么再加载什么。

就像你到一个新城市，不需要把整本旅游指南背下来。给你一张简明的地图（核心规则），然后告诉你“想了解这个景点的详细信息，翻到第 X 页”就够了。

💡 **渐进式披露的一个具体实现：Agent Skills**。Agent Skills 的核心机制就是“元数据常驻，正文按需加载”——每个 Skill 只在上下文中保留简短的名称和描述（几十个 Token），详细规则和执行流程只在触发时才动态注入推理上下文。这本质上和 OpenAI 的 AGENTS.md 当目录用是同一个思路，只不过 Skills 把这个模式进一步标准化了。详细介绍可以参考这篇：[Agent Skills 详解：是什么？怎么用？和 Prompt、MCP 有什么区别？](#)。

架构约束不能写在文档里，必须靠工具强制执行

他们给每个业务领域定义了固定的分层结构：

1

Types → Config → Repo → Service → Runtime → UI

依赖方向不能反过来。怎么保证？自定义 Linter 加结构测试。违反了就报错，报错消息里不光告诉你哪里错了，还直接告诉你怎么改。Agent 在被纠错的同时就被“教会”了正确的做法。

🔴 **OpenAI 原话**：If it cannot be enforced mechanically, agents will deviate.——文档中记录约束是不够的；如果不能机械化地强制执行，Agent 就会偏离。

可观测性也是给 Agent 看的，不只是给人看的

他们把 Chrome DevTools Protocol 接入了 Agent 运行时，Agent 能自己抓 DOM 快照、截图。日志、指标、链路追踪都通过本地可观测性栈暴露给 Agent。这样一来，“把启动时间降到 800ms 以下”就从一个模糊的愿望变成了 Agent 可以自己测量、自己验证的目标。

熵不会自己消失，必须主动对抗

一开始团队每周五花 20% 的时间手动清理 AI 生成物中的低质量代码。后来这事被自动化了——后台 Agent 定期扫描，找文档不一致、架构违规和冗余代码，自动提交清理 PR。清理的速度跟上了生成的速度，才能可持续地跑下去。

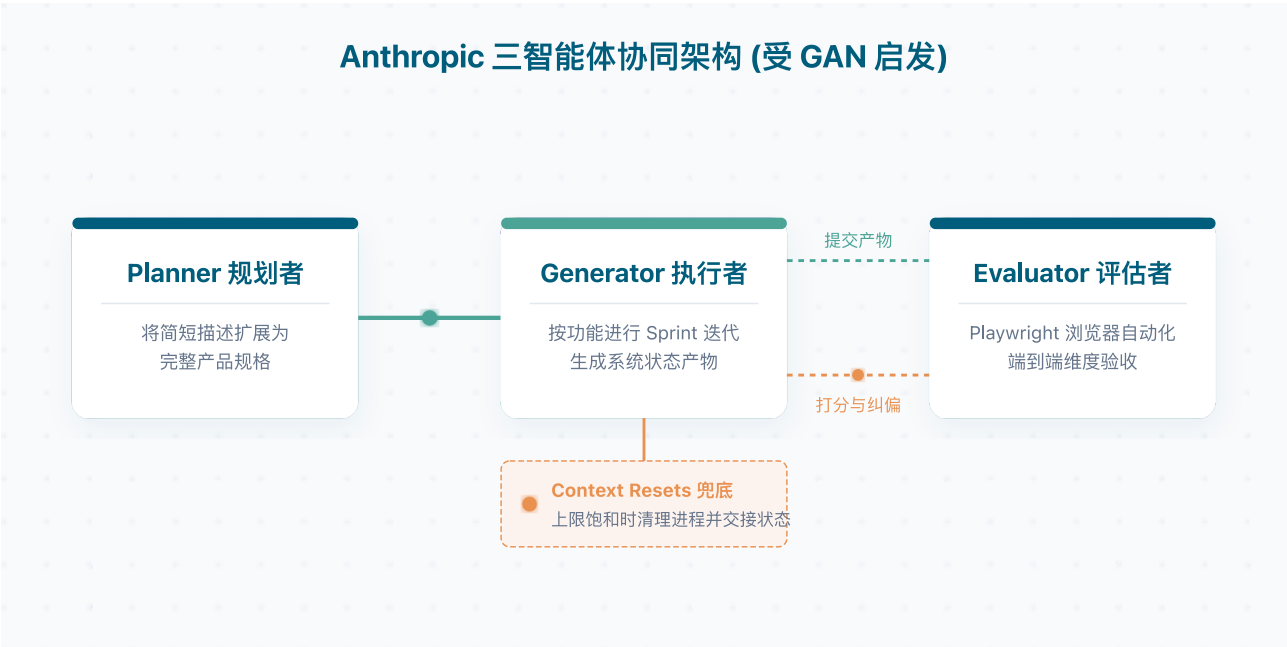
写在 Slack 里的知识，对 Agent 来说等于不存在

写在 Slack 讨论或 Google Docs 中的知识对 Agent 来说等于不存在。所有团队知识都作为版本控制的制品放置在仓库中。

⚠️ **工程视角**：OpenAI 自己也说了，这个结果“不应该被假设为在缺少类似投入的情况下可以复现”。他们的五大方法论每一项都需要大量前期投入，不要指望直接复制。但其中的**思维方式**（地图式文档、机械化约束、熵管理）是可以在任何规模上立即采用的。

Anthropic：从上下文焦虑到 GAN 式三智能体架构

Anthropic 在这个方向上有两个值得细看的实践，它们从不同角度揭示了 Harness 设计中容易被忽略的问题。



用 16 个 Agent 写了个 C 编译器，发现了什么？

Nicholas Carlini 用大约两周时间，跑了 16 个并行 Claude Opus 实例，大约 2000 个 Claude Code 会话，产出了一个 GCC torture test 通过率 99% 的 C 编译器。

指标	数值
持续时间	约 2 周
并行 Agent 数	16 个 Claude Opus 实例
会话数	约 2,000 个
产出	10 万行 Rust 代码
GCC torture test	99% 通过率
可编译项目	PostgreSQL、Redis、FFmpeg、CPython、Linux 6.9 Kernel 等 150+
API 成本	约 2 万美元

这个项目里几个 Harness 设计决策很有意思：

- 日志不往控制台打：全部写进文件，用 grep 友好的单行格式（`ERROR: [reason]`），主动控制上下文污染。

- **测试不全部跑**：每个 Agent 只跑随机 1-10% 的测试子集，但子采样对单个 Agent 是确定性的（同一次运行里每次都跑同样的子集），跨 VM 是随机的（不同 Agent 跑不同子集）。这样集体覆盖了全部测试，而单个 Agent 不会花几个小时在测试上打转。
- **Agent 角色专业化**：随着项目成熟，Agent 承担了专门角色——核心编译器工作、去重（LLM 生成的代码经常重新实现已有功能）、性能优化、代码质量和文档。

Carlini 后来说了一句很到位的话：“我必须不断提醒自己，我是在为 Claude 写这个测试框架，不是为自己写。”——**Harness** 的设计目标是让 **Agent** 高效工作，不是为了人类方便。

Anthropic 为什么要借鉴 GAN 的思路？

Anthropic Labs 团队在 2026 年 3 月发布了一个受 GAN（生成对抗网络）思路启发的三智能体架构（原文用的是"Taking inspiration from GANs"，是借鉴思路，不是真正的对抗训练）：



- **Planner**：拿到 1-4 句话的产品描述，扩展成完整的产品规格，被要求“在范围上要大胆”。
- **Generator**：按功能一个一个做"Sprint"，每个 Sprint 有明确的完成标准。
- **Evaluator**：用 Playwright MCP 实际点击运行中的应用，按产品设计深度、功能性、视觉设计、代码质量等维度打分。

这个架构要解决两个核心问题：

问题	表现	解法
上下文焦虑	Sonnet 4.5 快到上下文上限时草草收尾	context resets + 结构化交接（光靠压缩不够）
自我评价偏差	Agent 自信满满地夸自己做得好，实际质量一般	生成和评估交给两个独立的 Agent

打分标准本身也有讲究：前端设计方面，**设计质量和原创性的权重被故意调得比功能性和代码质量更高**——因为模型倾向于做出“功能齐全但长相平庸”的东西，权重调整是在引导它往更难的方向使劲。

遇到上下文焦虑，不是压缩而是重启

前面提到 Anthropic 发现 Sonnet 4.5 在上下文快填满时会出现“上下文焦虑”——变得犹豫、提前收工。光靠压缩上下文不够，他们的最终做法叫做 **context resets**（上下文重置）：

1. 当一个 Agent 的上下文接近饱和时，先把当前任务状态、已完成的工作、待办事项结构化地提取出来
2. 启动一个全新的“干净” Agent，把结构化的交接文档交给它
3. 新 Agent 从干净的状态继续工作

这就像程序碰到内存泄漏时的解法——你不去手动释放每一个内存块（对应上下文压缩），而是直接重启进程，从检查点恢复状态。虽然粗暴，但在长任务场景里，一个干净重启的 Agent 比一个塞满了历史信息的 Agent 表现好得多。

这个思路跟 Carlini 在编译器项目里的做法本质上是一回事——他跑了 2000 个 Claude Code 会话，每个会话都是独立的、从干净状态开始。只不过 Anthropic 把这个“重启-恢复”过程正式化和结构化了。

两种配置的成本对比：

配置	耗时	花费	效果
Solo Harness（单 Agent + 最少工具）	20 分钟	\$9	跑不起来的半成品
Full Harness（三 Agent + 完整工具链）	6 小时	\$200	完整可用的应用

更复杂的任务差距更明显——用 Full Harness 做一个浏览器里的音乐制作工作站（DAW），跑了将近 4 小时花了 \$124.70，产出了一个带有编曲视图、混音台和播放控制的可用程序。

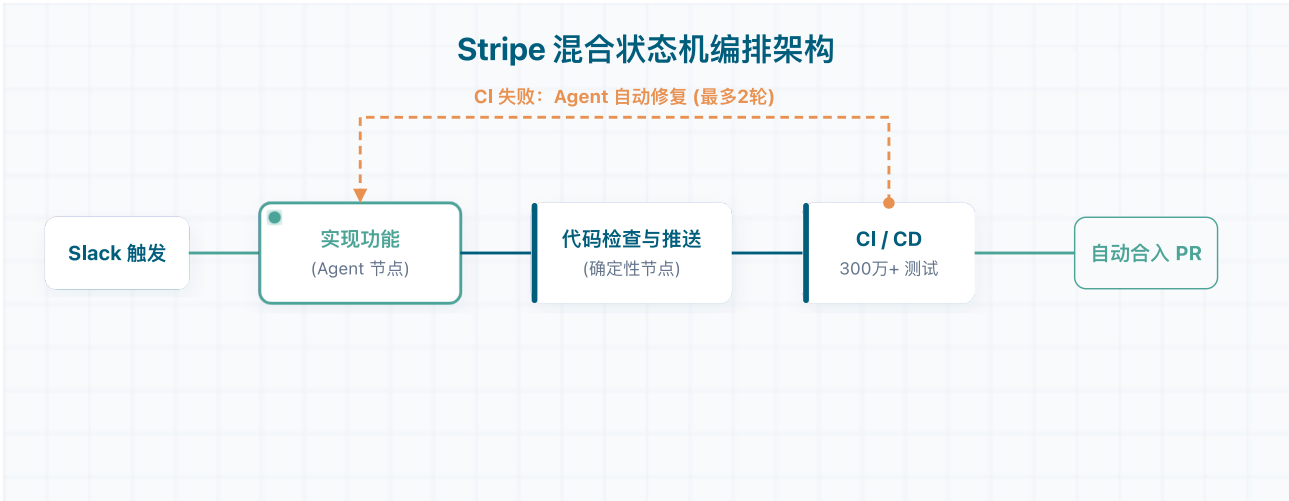
但有一个重要发现：当他们把模型从 Sonnet 4.5 换成 Opus 4.6 后，Sprint 机制可以完全移除，Evaluator 从每个 Sprint 检查变成了最后只做一次检查。

Anthropic 对此总结得非常精辟：**"Every component in a harness encodes an assumption about what the model can't do on its own, and those assumptions are worth stress testing."**（Harness 中的每个组件都编码了一个关于“模型靠自己做不到什么”的假设，而这些假设值得定期压力测试。）

🔴 **Anthropic 的结论：**"The space of interesting harness combinations doesn't shrink as models improve. Instead, it moves."——模型越强，不是不需要 Harness 了，而是 Harness 的设计空间转移到了新的位置。这意味着你需要**定期简化 Harness**——随着模型能力提升，之前必要的保护机制可能已经冗余了。

Stripe：每周 1300+ 个 PR，全程无人值守，他们是怎么做到的？

Stripe 的 Minions 系统代表了另一个极端——高度自动化的无人值守模式。开发者发一条 Slack 消息，Agent 就从写代码到跑 CI 到提 PR 全部搞定，人只在最后审查。每周超过 1300 个完全由 Minions 生产的、不含任何人写代码的 PR 被合并。



说实话，这个数字第一次看到的时候有点震惊。下面拆一下他们的架构。

组件	作用	关键设计
Devbox	开发环境	AWS EC2 预装源码和服务，预热池分配，启动约 10 秒，“牲口不是宠物”
编排状态机	流程控制	混合确定性节点（lint、push）和 Agent 节点（实现功能、修 CI），该确定的地方确定，该灵活的地方灵活
Toolshed MCP	工具服务	集中式 MCP 服务，近 500 个工具，每个 Minion 获得筛选子集
反馈回路	质量保障	Pre-push hook 秒级修 lint；推送后最多 2 轮 CI（300 万 + 测试）

Stripe 的编排设计思路很有意思。不是把所有事情都交给 Agent 判断，也不是全部走确定性流程，而是一个混合状态机——该确定的地方确定（跑 lint、推送代码），该灵活的地方灵活（实现功能、修 CI 错误）。就像一条工厂流水线，有些工位是机器人固定动作，有些工位是人工灵活处理。

📌 **核心理念：**"What's good for humans is good for agents."——为人类工程师投资的 Devbox、工具链和开发者体验，在 Agent 上也直接产生了回报。Agent 不是需要一套单独的基础设施，而是应该跟人类工程师用同一套，只是一开始就得被当作一等公民来设计。

Agent 底层是 Block 的开源 [goose](#) 项目的一个 fork，针对无人值守场景做了定制化。

Mitchell Hashimoto：不跑多 Agent，一个人的 Harness 工程学

Mitchell Hashimoto（Vagrant、Terraform、Ghostty 终端模拟器的作者）的实践路线和 Stripe 完全相反——他坚持一次只跑一个 Agent，保持深度参与。他明确说“我不打算跑多个 Agent，也不想跑”。

他的六步进阶路线：

步骤	名称	核心做法
1	放弃聊天模式	让 Agent 在能读文件、跑程序、发 HTTP 请求的环境里直接干活
2	复现自己的工作	每件事做两次——一次自己做，一次让 Agent 做，他形容“痛苦至极”
3	下班前启动 Agent	每天最后 30 分钟给 Agent 布置任务：深度调研、模糊探索、Issue 分拣
4	外包确定性任务	挑出 Agent 几乎一定能做好的任务后台跑着，建议关掉桌面通知避免上下文切换
5	工程化 Harness	每当 Agent 犯错，就工程化一个解决方案让它永远不再犯同样的错
6	始终有 Agent 在跑	目标是 10-20% 的工作时间有后台 Agent 运行

 **AGENTS.md 的正确用法：**Ghostty 项目里的 AGENTS.md，每一行都对应着一个过去的 Agent 失败案例。这不是写完就扔的静态文档，而是一个持续积累的防错系统——Agent 犯了一个新类型的错误，就加一行规则，以后就不会再犯了。

持续进化的 Harness 防错反馈闭环



Birgitta Böckeler 对 Harness 的系统化梳理

Birgitta Böckeler（Thoughtworks 的 Distinguished Engineer）在 Martin Fowler 网站上发表了对 OpenAI 实践的结构化分析。她的视角比较独特——不关注具体怎么做，而是关注这些做法可以归为哪几类、缺了什么。她把 Harness 组件归为三类：

归类	关注点	典型实践
Context Engineering	管理 Agent 看到什么、什么时候看到	从巨大 <u>AGENTS.md</u> 演化为入口文件 + 分层文档
Architectural Constraints	确保 Agent 不跑偏	自定义 Linter、结构测试、LLM Agent 充当约束
Garbage Collection	对抗熵积累	定期运行清理 Agent 扫描不一致和违规

Böckeler 还提了几个挺有前瞻性的判断：

1. **Harness 将成为新的服务模板**——大多数组织只有两三个主要技术栈，未来团队可能会从一组预制 Harness 中选择，就像今天从服务模板实例化新服务一样。
2. **棕地项目改造是最大挑战**——所有公开成功案例都是绿地项目，将有十年历史、没有架构约束的代码库引入 Harness Engineering 是更复杂的问题。Böckeler 把它比作“在从未用过静态分析工具的代码库上运行静态分析——你会被警报淹没”。她还提出了一个关键概念“*Ambient Affordances*”：强类型语言天然有类型检查作 sensor，清晰的模块边界方便定义架构约束，Spring 这样的框架抽象了很多细节——**环境本身的结构特性决定了 Harness 能做多好。**
3. **功能验证体系几乎缺席**——大量讨论了架构约束和熵管理，但功能正确性验证是被严重忽视的领域。Böckeler 对此有一个更尖锐的观察：很多团队只是让 AI 生成测试套件然后看它是否绿色通过，但这“puts a lot of faith into AI-generated tests, that's not good enough yet”——用 AI 生成的测试来验证 AI 生成的代码，本质上是在用同一双眼睛检查自己的作业。

推荐阅读：

- [OpenAI - Harness Engineering: Leveraging Codex in an Agent-First World](#)
- [Anthropic - Harness Design for Long-Running Application Development](#)
- [Mitchell Hashimoto - My AI Adoption Journey](#)
- [Birgitta Böckeler - Harness Engineering \(Martin Fowler 网站\)](#)
- [Stripe - Minions: Stripe's One-Shot, End-to-End Coding Agents](#)
- [LangChain - The Anatomy of an Agent Harness](#)
- [Can Bölük \(Can.ac\) - The Harness Problem](#)
- [Harness Engineering 深度解析：AI Agent 时代的工程范式革命](#)
- [一文看懂 Harness engineering：智能体时代的 AI 编程驾驭之道](#)

最近更新: 2026/4/15 11:45

贡献者: Guide